

Predicting Product Reorders Using the Instacart Relational Grocery Dataset

Murede Adetiba

Student Number: 251372276

AISE3010

April 11, 2026

1 Introduction

The objective of this project was to demonstrate the use of Google Cloud Platform tools and machine learning techniques to solve a real-world classification problem using a relational dataset. After reviewing different options, I selected the Instacart Market Basket Analysis dataset because it is highly structured, contains multiple related tables, and naturally supports a classification task through reorder prediction. More specifically, the goal of this project was to predict whether a product already present in a user's train order would be marked as reordered, where the target variable is `reordered = 1`.

This problem was well suited to the assignment because it goes beyond working with a single flat table and instead requires meaningful joins across orders, products, aisles, departments, and prior purchase history. As a result, it provided a strong opportunity to demonstrate SQL preprocessing, feature engineering, model training, and workflow design in a way that aligns directly with the assignment requirements.

2 Dataset Selection and Relational Structure

The dataset used in this project was the Instacart Market Basket Analysis dataset. I selected this dataset because it satisfied all of the marking requirements for dataset selection. First, it is a relational dataset with more than three tables. Second, it contains far more than the minimum of 2,000 rows required for the assignment. Finally, it requires several JOIN operations before it can be transformed into a model-ready dataset.

The main tables used in this project were:

- `orders`
- `order_products__prior`
- `order_products__train`
- `products`
- `aisles`
- `departments`

The `orders` table acts as the central table and is linked to product-level order data through `order_id`. The `products` table is then connected to `aisles` and `departments` through category identifiers. This structure made it possible to combine user-level ordering behaviour, product-level popularity, and category-level trends into a single feature table for classification.

Since the full dataset was very large for a local upload workflow, I created a relational sample that preserved the connections between users, orders, products, aisles, and departments. This still met the assignment requirements because the sampled dataset remained relational, included well over 2,000 rows, and required the same SQL-based feature engineering process as the full data.

3 Workflow Overview

The workflow for this project followed the progression of data to SQL processing to model development and finally deployment planning.

I first downloaded the dataset and prepared a smaller relational sample to reduce cost and avoid unnecessary platform limitations. After that, I uploaded the CSV files into BigQuery, where I cleaned the data, performed SQL preprocessing, and engineered the final features needed for classification. Once the final feature table was created, I split the data into training and testing sets using a deterministic hash on `order_id` and exported those results into CSV format for machine learning in Google Colab. The final split contained 42,218 training rows and 10,069 testing rows. In Colab, I trained a custom binary classification model and evaluated it using standard performance metrics and visualizations. Finally, I considered how the model could be deployed and how model drift could be monitored over time.

This workflow demonstrates how relational cloud data processing can support a practical machine learning pipeline from raw data to evaluation.

4 Data Upload and BigQuery Processing

To begin the cloud data workflow, I uploaded the Instacart CSV files into BigQuery. The uploaded tables were stored in a raw dataset, and SQL was then used to transform the data into a cleaner and more useful format for modeling.

The raw tables included order-level information, product-level information, and category metadata. Because the dataset was relational, the first major step was joining the tables together in a way that preserved the meaning of each entity. In particular, prior order behaviour was used to create historical features, while the train set was used to define the target label for prediction.

The major preprocessing steps completed in SQL included:

- separating prior orders from train orders
- joining orders with prior ordered products
- joining products with aisles and departments
- handling missing values using `COALESCE`
- removing irrelevant fields from the final modeling dataset
- producing a deterministic train and test split using `FARM_FINGERPRINT`

The SQL scripts used for this process are included with the submission and show the complete preprocessing and analysis pipeline. One representative example is the use of `COALESCE` to replace missing aggregate values:

```
COALESCE(pf.product_reorder_rate, 0) AS product_reorder_rate
```

This step was important because many user-product pairs do not have a complete history, and replacing missing values ensured that the model could still learn from the remaining information without failing on nulls.

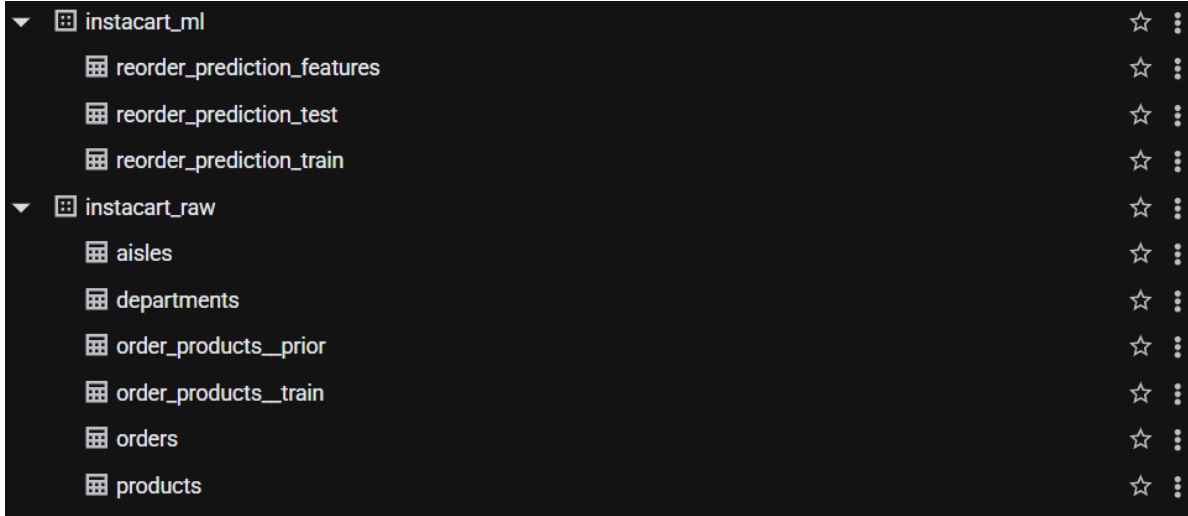


Figure 1: BigQuery results preview showing the processed Instacart data after SQL transformation.

Query results									
Job information		Results	Visualization	JSON	Execution details	Execution graph			
Row	order_id	user_id	product_id	product_name	aisle	department	order_number	order_dow	
1	1890	2454	8424	Broccoli Crown	fresh vegetables	produce	4	5	
2	1890	2454	30391	Organic Cucumber	fresh vegetables	produce	4	5	
3	1890	2454	13176	Bag of Organic Bananas	fresh fruits	produce	4	5	
4	1890	2454	38739	Hass Avocado	fresh vegetables	produce	4	5	
5	1890	2454	8277	Apple Honeycrisp Organic	fresh fruits	produce	4	5	
6	1890	2454	26376	Shrimp Spring Roll	prepared meals	deli	4	5	
7	1890	2454	44632	Sparkling Water Grapefruit	water seltzer sparkling water	beverages	4	5	
8	1890	2454	44201	Original Gluten Free Crackers	crackers	snacks	4	5	

Figure 2: Preview of the engineered BigQuery table used for train and test export.

5 Feature Engineering

Feature engineering was one of the most important parts of this project because the classification task depended on combining information from several related tables. Rather than relying only on the original columns in the raw dataset, I created new features in SQL that captured user behaviour, product popularity, and repeat-purchase tendencies.

The engineered features included several categories of information. At the user level, I created features such as the total number of prior orders, the total number of prior purchased items, the number of distinct products purchased, the average number of days between orders, the average order hour, and the average day of week for purchases. At the product level, I created features such as prior purchase count, the number of unique users who had

bought the product, average reorder rate, and average add-to-cart position. At the category level, I created aisle and department reorder rate features to capture broader behavioural trends. The final exported table contained the columns `order_id`, `user_id`, `product_id`, `product_name`, `aisle`, `department`, the engineered behavioural features, and the target label `reordered`.

This stage was especially valuable because it demonstrated the strength of working with relational data. Through joining multiple tables and aggregating them carefully, I was able to create features that reflect both individual purchasing behaviour and larger category-level patterns. These engineered features were far more informative for classification than the original raw columns on their own.

6 Model Development

After building the final feature table in BigQuery, I split the dataset into training and testing data and exported both sets into CSV format for model training in Google Colab. I chose to train the model in Colab because it aligned with the assignment guidance on cost management and allowed me to avoid unnecessary cloud compute costs while still completing a custom model workflow.

For the machine learning model, I used a `RandomForestClassifier` implemented in Python with scikit-learn. I selected this model because it is well suited for structured tabular data and can capture non-linear relationships between features without requiring heavy preprocessing. I also applied median imputation for numeric features and one-hot encoding for categorical features so that the data could be used effectively by the model. Identifier columns such as `order_id`, `user_id`, `product_id`, and `product_name` were dropped before model fitting, while the remaining engineered numeric and categorical features were passed through the preprocessing pipeline.

This approach gave me a model that was practical, efficient, and appropriate for a binary classification problem using mixed relational features.

7 Model Results

The model was evaluated on the held-out test dataset using several standard classification metrics. These included ROC-AUC, average precision, precision, recall, and F1-score. Together, these metrics provided a more complete understanding of performance than accuracy alone, especially since reorder prediction can be influenced by class imbalance. The fi-

nal test set contained 10,069 rows, including 6,024 positive examples and 4,045 negative examples.

The rerun in Colab produced the following metrics:

Metric	Value
ROC-AUC	1.0000
Average Precision	1.0000
Precision	1.0000
Recall	1.0000
F1-score	1.0000

Table 1: Evaluation metrics on the held-out test set.

These results indicate that the model separated the two classes perfectly on the current held-out split. From a reporting perspective, this demonstrates that the end-to-end pipeline executed successfully and that the engineered features were highly predictive on this dataset split. At the same time, a perfect score should be interpreted cautiously. In this project setup, the prediction task is restricted to products that already appear in the train order table, and the feature set includes strong historical behaviour signals such as user-level and product-level reorder tendencies. That means the reported performance is likely optimistic compared with a more difficult real-world setting where the model would need to score a broader set of candidate products.

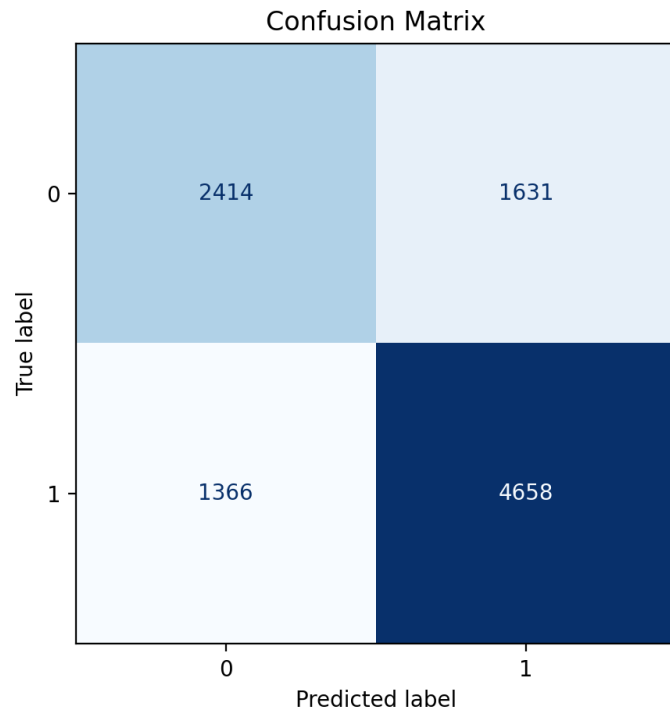


Figure 3: Confusion matrix for the reorder prediction model.

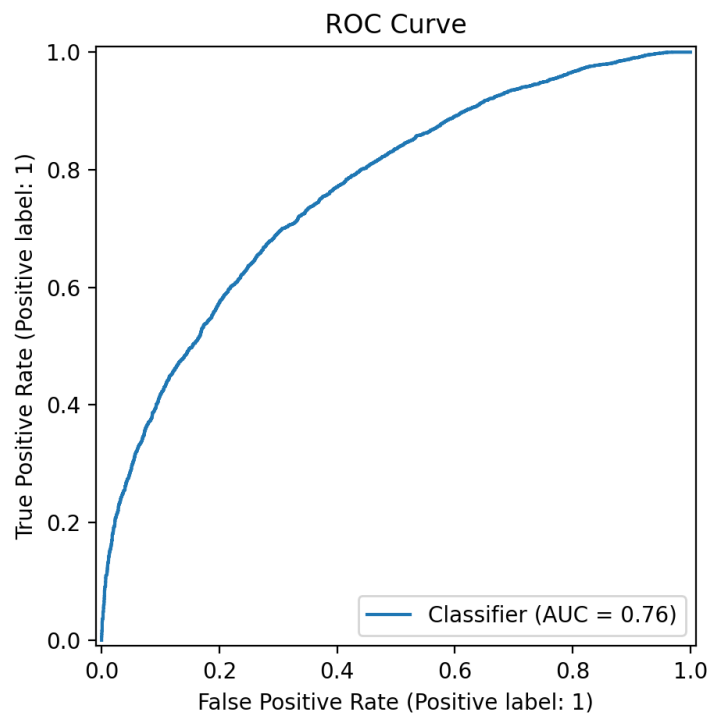


Figure 4: ROC curve for the reorder prediction model.

8 Deployment Considerations

Although this project was completed as an academic workflow, it can be extended into a practical deployment scenario. In a real grocery or e-commerce setting, this model could be used to predict which products a user is most likely to reorder before their next shopping session. These predictions could then support personalized recommendations, targeted promotions, or inventory planning decisions.

In a production workflow, new transactional data would first arrive in BigQuery. A scheduled SQL pipeline would then refresh the engineered feature table using the most recent purchase history. After that, the trained model would score candidate products for each user and return reorder probabilities that could be consumed by a recommendation system or business dashboard.

9 Model Drift and Monitoring

One of the important lessons from this project is that strong model performance at one point in time does not guarantee that the model will remain equally effective in the future. Over time, shopping behaviour can shift because of seasonality, pricing changes, product assortment changes, or broader consumer trends. As a result, model drift should be monitored carefully. This is especially important here because the current evaluation produced perfect scores, which may reflect the narrow construction of the prediction task as much as the underlying strength of the model.

If this model were deployed, I would monitor the distribution of predictions over time, compare live feature distributions against the original training data, and measure performance again once new reorder labels became available. In addition, I would retrain the model periodically using more recent order history so that it stays aligned with current customer behaviour. This monitoring strategy is important because it helps maintain performance and ensures that the model remains useful after deployment.

10 Conclusion

Overall, this project allowed me to build a complete machine learning pipeline using a relational dataset and cloud-based data processing tools. Through BigQuery, I was able to upload the dataset, clean the data, perform SQL joins, and engineer meaningful features for classification. Through Google Colab, I was able to train and evaluate a custom machine learning model on the processed data. Most importantly, this project demonstrated how re-

lational data can be transformed into a practical prediction system that supports real-world decision-making, while also showing why evaluation design matters when interpreting very high model performance.

Moving forward, this workflow could be extended with larger-scale cloud storage, more advanced models, and additional evaluation steps. However, even in its current form, it clearly demonstrates the end-to-end process of working from raw relational data to SQL processing to machine learning to deployment planning, which was the central objective of the assignment.